C# & .NET Exam Paper 3: OOP & Integrated Software Systems

Comprehensive Test covering Classes, Constructors, Methods, Encapsulation, and System Integration

Duration: *90 Minutes* | ***Total Marks:*** *70 Marks* | ***Format:*** *Written & Practical Coding*

Section A: Object-Oriented Fundamentals & Class Design (20 Marks)

Demonstrate knowledge of OOP encapsulation, fields, properties, and access modifiers.

Q1. Which access modifier restricts access to members so they are only visible within the containing class? [3 Marks]

A) public

B) private

C) protected

D) internal

Q2. Explain why backing fields should be private while expose auto-properties with { get; set; } in C#. [7 Marks]

Q3. Write a C# class named Book with private fields for title, author, and price. Provide a constructor, public auto-properties, and a PrintDetails() method. [10 Marks]

Section B: Constructors & Instance Methods (20 Marks)

Evaluates object instantiation, instance method design, and object state validation.

Q4. Design a BankAccount class with a Balance property. Ensure that the Deposit(double amount) method only accepts positive values, and Withdraw(double amount) prevents overdrawing. [20 Marks]

Section C: End-to-End System Integration Project (30 Marks)

Combines Classes, File handling (System.IO), and List collections in a single mini-application.

Q5. Write a complete C# application that reads product records from "products.txt" (format: Name,Price), creates Product objects, stores them in a List, filters products priced over $50.00, and prints the filtered list to the Console. [30 Marks]

EXAM ANSWER KEY & MARKING SCHEME

Comprehensive Solutions & Grading Criteria

Solutions: Section A: Object-Oriented Fundamentals & Class Design (20 Marks)

Q1. Solution & Criteria:

Correct Answer: B. private hides members from external classes, enforcing encapsulation.

Q2. Solution & Criteria:

Private backing fields prevent unauthorized external tampering. Auto-properties maintain encapsulation, allow validation logic inside getters/setters, and enable data-binding frameworks.

Q3. Solution & Criteria:

```
public class Book {
public string Title { get; set; }
public string Author { get; set; }
public double Price { get; set; }

public Book(string title, string author, double price) {
Title = title;
Author = author;
Price = price;
}

public void PrintDetails() {
Console.WriteLine($"'{Title}' by {Author} - ${Price:F2}");
}
}
```

Solutions: Section B: Constructors & Instance Methods (20 Marks)

Q4. Solution & Criteria:

```
public class BankAccount {
public string AccountNumber { get; }
public double Balance { get; private set; }

public BankAccount(string accNum, double initialBalance) {
AccountNumber = accNum;
Balance = initialBalance > 0 ? initialBalance : 0;
}

public void Deposit(double amount) {
if (amount > 0) Balance += amount;
}

public bool Withdraw(double amount) {
if (amount > 0 && amount <= Balance) {
Balance -= amount;
return true;
}
return false;
}
}
```

Solutions: Section C: End-to-End System Integration Project (30 Marks)

Q5. Solution & Criteria:

```
using System;
using System.Collections.Generic;
using System.IO;

class Product {
public string Name { get; set; }
public double Price { get; set; }
public Product(string name, double price) {
Name = name;
Price = price;
}
}

class Program {
static void Main() {
List<Product> productList = new List<Product>();
if (!File.Exists("products.txt")) return;

string[] lines = File.ReadAllLines("products.txt");
foreach (string line in lines) {
string[] parts = line.Split(',');
if (parts.Length == 2) {
string name = parts[0].Trim();
if (double.TryParse(parts[1].Trim(), out double price)) {
productList.Add(new Product(name, price));
}
}
}

Console.WriteLine("--- Products Over $50 ---");
foreach (Product p in productList) {
if (p.Price > 50.00) {
Console.WriteLine($"{p.Name}: ${p.Price:F2}");
}
}
}
}
```